

学习记录

一、在Pycharm里面接入大模型进行使用

豆包给出的在Pycharm里部署大模型的几种方法

1.调用 豆包/火山大模型

需要申请API key 太麻烦了，直接淘汰

API是啥？：

软件和软件之间的接口。

2.下载Ollama，调用本地大模型

- 容易部署
- 可以本地运行
- 并且本地大模型够用了

下载Ollama

1.啥是ollama?

Ollama可以下载和调用各种本地大模型如千问、Deepseek等等

2.下载ollama

豆包说的在ollama.com官方下载，但是由于是国外网站下载太慢了。

豆包又说在ollama.ac.cn国内镜像网站下载，但是不知道为什么打不开

最终我发现应用宝里就可以下载，又快又简单。

3.下载需要的大模型

在ollama官网搜索deepseek，复制指令

deepseek-v4-flash

↓ 25.5K Downloads ⌚ Updated 4 days ago

DeepSeek-V4-Flash is a preview of the DeepSeek-V4 series, a Mixture-of-Experts model with 284B total parameters and 13B activated, built for efficient reasoning across a 1M-token context window.

[tools](#) [thinking](#) [cloud](#)

[CLI](#) [cURL](#) [Python](#) [JavaScript](#)

```
ollama run deepseek-v4-flash:cloud
```



Applications



Claude Code

```
ollama launch claude --model deepseek-v4-flash:cloud
```



Codex

```
ollama launch codex --model deepseek-v4-flash:cloud
```



OpenCode

```
ollama launch opencode --model deepseek-v4-flash:cloud
```



OpenClaw

```
ollama launch openclaw --model deepseek-v4-flash:cloud
```



Hermes Agent

```
ollama launch hermes --model deepseek-v4-flash:cloud
```



打开Windows Powershell，输入指令下载deepseek

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

PS C:\Windows\System32> ollama run deepseek-v4-flash:cloud
```

4.在pycharm里接入deepseek



在pycharm的终端里面输入这样的指令就可以了。在编程的时候ollama就可以被当作一个函数库来调用。

豆包写的代码示例：

- ollama.chat()的用法：这个用于多次对话，可以有历史记忆。

```
import ollama

# 调用 ollama.chat 发送对话
response = ollama.chat(
    model="llama3.1:8b", # 必须写：用哪个模型
    messages=[           # 必须写：对话历史
        {"role": "user", "content": "你好"}
    ]
)

# 打印 AI 回答
print(response["message"]["content"])
```

```
import ollama
def interactive_chat():
    messages = [] # 保存对话历史
    print("🤖 欢迎使用 ollama 聊天机器人！输入 'quit' 退出。\\n")

    while True:
        user_input = input("你: ")
        if user_input.lower() == "quit":
            print("🤖 再见！")
            break

        # 将用户消息加入历史
        messages.append({"role": "user", "content": user_input})

        # 调用模型
        response = ollama.chat(
            model="deepseek-r1:8b",
            messages=messages
        )
        reply = response["message"]["content"]

        # 将模型回复加入历史
        messages.append({"role": "assistant", "content": reply})
        print(f"🤖: {reply}\\n")
    interactive_chat()
```

🤖 欢迎使用 Ollama 聊天机器人！输入 'quit' 退出。

你: 我今天心情不好

🤖: 听到你心情不好，我很想抱抱你（*虚拟拥抱*）。能和我聊聊是什么让你感到低落吗？或者只是想说说也没关系。有时候把情绪说出来，哪怕只是零碎的感受，心情也会轻松一些。

如果你愿意，可以告诉我最近发生了什么，或者我们也可以聊聊别的事情，比如你喜欢的电影、音乐，或者你喜欢的小动物、美食.....任何话题都可以。

- ollama.generate()的用法：AI直接给你回答不能对话。

```
response = ollama.generate(  
    model="llama3.1:8b",    # 模型  
    prompt="给我写个标题", # 提示词  
)
```

二、用接入的大模型来写一个可以检查代码的函数

我的想法：写一个函数可以在写代码过程中直接，一键检查之前的代码，不用复制代码

1.首先需要一个工具可以直接复制函数到容器里

豆包给了几个方案：

- 方案一：

```
with open(__file__,"r",encoding="utf-8") as f: #open是打开文件的内置函数，“r”代表只读模式，as f表示把读取到的内容赋到f这个变量  
    code = f.read() #f.read是指一次性读取文件的全部内容  
    #其他：f.close返回None关闭文件，f.closed返回布尔值判断文件是否关闭
```

这个函数可以复制整个文件里面的所有代码到code这个容器里面，但是会把所有代码复制进去，不够精确。淘汰。

- 方法二：

```
def get_code(start_line:int,end_line:int)->str:#返回值是str类型  
    with open(__file__,"r",encoding="utf-8") as f:#encoding指的是用指定文件字符编码格式参数，告诉python用什么编码规则去读取，utf-8是一种字符编码规则  
        lines = f.readlines()#指按行读取文件中的内容返回列表，f.readline返回字符串。  
        selected_lines = lines[start_line-1:end_line]  
        return"".join(selected_lines) #"".join(selected_lines)可以把列表里的所有字符串拼接成一个完整的字符串
```

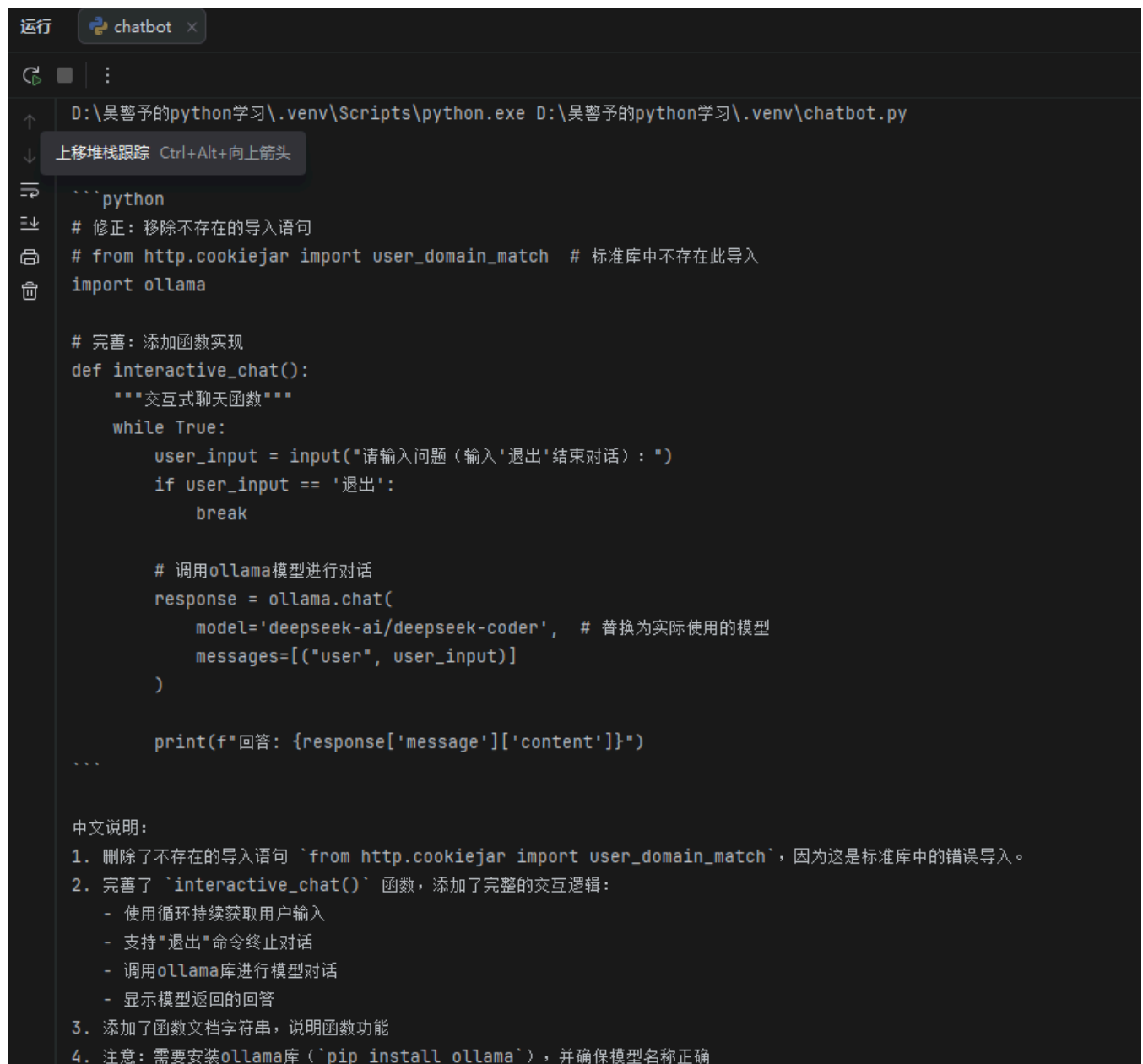
这个函数可以精确的读代码，很适合我的要求

2.编写代码

```
def get_code(start_line:int, end_line:int) -> str:#先写这个函数来实先复制函数的功能  
    with open(__file__,"r",encoding="utf-8") as f:  
        lines = f.readlines()  
        lines_selected= lines[start_line-1:end_line]  
        return "".join(lines_selected)  
def code_checker(start_line:int, end_line:int) :  
    code = get_code(start_line, end_line)#获取要检查的代码  
    reply = ollama.generate(  
        model="deepseek-r1:8b",#调用deepseek  
        prompt=f"给我检查这段代码,并且修改，说中文：\n{code}",  
    )  
    print(reply["response"])
```

```
code_checker(,)
```

运行后的样子：



```
运行 chatbot x
D:\吴警予的python学习\.venv\Scripts\python.exe D:\吴警予的python学习\.venv\chatbot.py
上移堆栈跟踪 Ctrl+Alt+向上箭头

```python
修正：移除不存在的导入语句
from http.cookiejar import user_domain_match # 标准库中不存在此导入
import ollama

完善：添加函数实现
def interactive_chat():
 """交互式聊天函数"""
 while True:
 user_input = input("请输入问题（输入'退出'结束对话）：")
 if user_input == '退出':
 break

 # 调用ollama模型进行对话
 response = ollama.chat(
 model='deepseek-ai/deepseek-coder', # 替换为实际使用的模型
 messages=[("user", user_input)]
)

 print(f"回答: {response['message']['content']}")
...

中文说明：
1. 删除了不存在的导入语句 `from http.cookiejar import user_domain_match`，因为这是标准库中的错误导入。
2. 完善了 `interactive_chat()` 函数，添加了完整的交互逻辑：
 - 使用循环持续获取用户输入
 - 支持"退出"命令终止对话
 - 调用ollama库进行模型对话
 - 显示模型返回的回答
3. 添加了函数文档字符串，说明函数功能
4. 注意：需要安装ollama库（`pip install ollama`），并确保模型名称正确
```